

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Materia: Programación 1

Carrera: Tecnicatura Universitaria en Programación

Institución: Universidad Tecnológica Nacional (UTN)

Integrantes del equipo

Integrante 1 · Joaquin Cano

Integrante 2 · Hernan Araujo

Fecha de entrega: Junio 2025

1. Marco Teórico	3
1.1 Listas en Python	3
1.2 Diccionarios en Python	3
1.3 Funciones	4
1.4 Estructuras Condicionales	4
1.5 Estructuras Repetitivas	4
1.6 Ordenamientos	5
1.7 Estadísticas Básicas	5
1.8 Archivos CSV	5
2. Decisiones Técnicas y Arquitectura	6
2.1 Estructura del proyecto	6
2.2 Diagrama de flujo	6
2.3 Capturas de ejecución	7
3. Dificultades y Conclusiones	8
4. Bibliografía / Webgrafía	9

1. Marco Teórico

1.1 Listas en Python

Una **lista** es una estructura de datos mutable y ordenada que permite almacenar colecciones heterogéneas de elementos. En Python se define con corchetes y los elementos se acceden mediante índices numéricos que comienzan en cero. Las listas soportan operaciones de inserción, eliminación, búsqueda y recorrido mediante bucles, lo que las convierte en la estructura principal de almacenamiento en memoria de este proyecto.

En el sistema de gestión de países, cada país se representa internamente como un diccionario, y todos los países se agrupan en una única lista llamada `países`. Esta lista es cargada desde el archivo CSV al inicio de la sesión y es el dato de trabajo de todas las funciones del sistema.

```
países = [] # lista vacía
países.append({'nombre': 'Argentina', ...}) #
agregar elemento
len(países) # cantidad de elementos
```

1.2 Diccionarios en Python

Un **diccionario** es una estructura de datos que almacena pares clave-valor. Permite acceder a los datos mediante claves descriptivas en lugar de índices numéricos, lo que hace el código más legible y menos propenso a errores. En Python se define con llaves (`{ }`) y es mutable.

Cada país del sistema se representa como un diccionario con cuatro claves: `nombre`, `poblacion`, `superficie` y `continente`. Este diseño facilita el acceso semántico a cada atributo sin depender del orden de los campos.

```
pais = { 'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400,
        'continente': 'América' } print(pais['nombre']) # → Argentina
```

1.3 Funciones

Una **función** es un bloque de código reutilizable que realiza una tarea específica y puede recibir parámetros de entrada y retornar valores. El principio de diseño una función = una responsabilidad (principio de responsabilidad única) garantiza que cada función sea independiente, fácil de probar y de mantener.

El sistema aplica este principio de forma sistemática: `cargar_paises()` únicamente lee el CSV; `guardar_paises()` únicamente escribe; `agregar_pais()` gestiona el alta; `filtrar_paises()` aplica filtros; y así sucesivamente. Las funciones utilitarias como `pedir_entero()` y `normalizar_texto()` se reutilizan en múltiples contextos.

1.4 Estructuras Condicionales

Las **estructuras condicionales** (`if` / `elif` / `else`) permiten que el programa tome decisiones en función del valor de una expresión booleana. Son fundamentales para implementar validaciones, menús de opciones y bifurcaciones de lógica de negocio.

UTN — Tecnicatura Universitaria en Programación | Programación 1 Página 3

El sistema utiliza condicionales en múltiples lugares: para validar que los campos ingresados no estén vacíos, para detectar duplicados al agregar un país, para seleccionar el criterio de filtrado o de ordenamiento, y para mostrar mensajes de éxito o error al usuario.

```
if not nombre: print('[ERROR] El nombre no puede estar vacío.') return if
any(p['nombre'].upper() == nombre.upper() for p in paises): print('[ERROR]
Ya existe ese país.')
```

1.5 Estructuras Repetitivas

Las **estructuras repetitivas** (`for` y `while`) permiten ejecutar un bloque de código múltiples veces. El bucle `for` itera sobre secuencias de longitud conocida, mientras que `while` se utiliza cuando la condición de corte depende de la entrada del usuario.

El menú principal del sistema emplea un bucle `while True` que mantiene el programa activo hasta que el usuario elige la opción de salida. Los recorridos sobre la lista de países

(para filtrar, calcular estadísticas o mostrar resultados) emplean bucles `for`. La función `pedir_entero()` usa `while True` para reintentar la entrada en caso de error.

1.6 Ordenamientos

El **ordenamiento** es el proceso de reorganizar una colección de elementos según un criterio específico. Python provee la función incorporada `sorted()`, que devuelve una nueva lista ordenada sin modificar la original, y acepta los parámetros `key` (función que extrae el valor de comparación) y `reverse` (booleano para orden descendente).

El sistema permite ordenar el listado de países por nombre, población o superficie, tanto en forma ascendente como descendente. El parámetro `key=lambda p: p[clave]` indica qué campo del diccionario se usa como criterio de comparación.

```
ordenados = sorted(paises, key=lambda p: p['poblacion'], reverse=True)
```

1.7 Estadísticas Básicas

Las **estadísticas descriptivas** permiten resumir y comprender un conjunto de datos numéricos. Las métricas implementadas son: **máximo** (país con mayor población/superficie), **mínimo** (país con menor población), **promedio o media aritmética** (suma de valores dividida por la cantidad) y **conteo por categoría** (cantidad de países por continente).

Python ofrece las funciones `max()`, `min()` y `sum()` que, combinadas con la función `key`, operan directamente sobre listas de diccionarios sin necesidad de librerías externas.

```
max_pob = max(paises, key=lambda p: p['poblacion'])
prom_pob = sum(p['poblacion'] for p in paises) / len(paises)
```

1.8 Archivos CSV

Un archivo **CSV** (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos están separados por comas. Es el estándar más extendido para el intercambio de datos tabulares por su simplicidad y compatibilidad con hojas de cálculo

y bases de datos.

Python incluye el módulo `csv` en su biblioteca estándar. La clase `csv.DictReader` lee el archivo y convierte cada fila en un diccionario cuyas claves son los encabezados de la primera línea. Esto permite acceder a cada campo por nombre en lugar de por índice. La clase `csv.DictWriter` realiza la operación inversa para persistir los cambios.

```
with open('paises.csv', newline='', encoding='utf-8') as f: lector =
    csv.DictReader(f)
    for fila in lector:
        paises.append({'nombre': fila['nombre'],
            ...})
```

2. Decisiones Técnicas y Arquitectura

2.1 Estructura del Proyecto

El proyecto se organiza en un único archivo Python (`main.py`) y un archivo de datos (`países.csv`). La modularización se logra mediante funciones claramente delimitadas, cada una con una única responsabilidad. Las dependencias entre funciones son mínimas y explícitas: todas reciben la lista `países` como parámetro.

Tabla de funciones del sistema

cargar_paises(ruta)	Lee el CSV y devuelve la lista de países
guardar_paises(paises, ruta)	Persiste la lista en el CSV
agregar_pais(paises)	Alta de un nuevo país con validaciones
actualizar_pais(paises)	Modifica población y/o superficie
buscar_pais(paises)	Búsqueda parcial o exacta por nombre
filtrar_paises(paises)	Filtra por continente, población o superficie
ordenar_paises(paises)	Ordena por nombre, población o superficie
mostrar_estadisticas(paises)	Calcula y muestra métricas estadísticas
mostrar_lista(paises, titulo)	Imprime la tabla formateada de países
pedir_entero(mensaje)	Lee y valida un entero positivo del usuario
pedir_entero_opcional(...)	Como pedir_entero pero acepta Enter para mantener
normalizar_texto(texto)	Elimina tildes y convierte a minúsculas para comparar
buscar_exacto(paises, nombre)	Retorna el diccionario del país si existe exacto
menu()	Bucle principal del menú de consola

2.2 Diagrama de Flujo — Operaciones Principales

A continuación se representa el flujo de ejecución del sistema:

INICIO
Cargar paises.csv → lista paises[]
Mostrar menú principal
¿Opción elegida por el usuario?

1: Agregar 2: Actualizar 3: Buscar
4: Filtrar 5: Ordenar 6: Estadísticas 0: Salir
Ejecutar función correspondiente
Validar entrada → si hay error → mostrar mensaje y volver al menú

Búsqueda por nombre parcial — término 'ar':

```

■■■ Buscar país ■■■ Nombre (parcial o exacto): ar Resultados para 'ar' (3
resultado/s) Nombre Población Superficie Continente 
Argentina 45.376.763
2.780.400 km² América Qatar 2.930.524 11.586 km² Asia Madagascar 28.915.653
587.041 km² África

```

Estadísticas generales:

■ Estadísticas ■ Total de países: 20 Mayor población: China (1.412.600.000)
Menor población: Islandia (370.000) Promedio de población: 108.432.500
Promedio de superficie: 1.243.870 km² Países por continente: África: 4
América: 5 Asia: 5 Europa: 5 Oceanía: 1

Filtro por rango de superficie:

[illegible]

3. Dificultades y Conclusiones

3.1 Dificultades técnicas encontradas

Durante el desarrollo del proyecto surgieron las siguientes dificultades:

- **Manejo de codificación UTF-8 en el CSV:** al abrir el archivo sin especificar

`encoding='utf-8'`, los caracteres especiales como tildes y la letra ñ generaban errores de lectura. La solución fue agregar el parámetro de codificación explícitamente en todas las operaciones de apertura de archivo.

- **Normalización de texto para comparaciones:** comparar cadenas con tildes (por ejemplo 'América' vs 'America') requirió implementar la función `normalizar_texto()`, que utiliza el módulo `unicodedata` para eliminar diacríticos antes de comparar.

- **Validación de entradas numéricas:** el método `str.isdigit()` no acepta el signo negativo, por lo que fue necesario diseñar un bucle de reintento que garantice que el usuario ingrese siempre un entero positivo.

- **Modularización del menú de filtros:** la función `filtrar_paises()` debió manejar tres sub-opciones con lógicas distintas. Se optó por mantenerlas dentro de la misma función para simplificar la estructura, siendo conscientes de que en un proyecto mayor convendría separar cada filtro en su propia función.

3.2 Conclusiones

El desarrollo de este trabajo permitió integrar de forma práctica todos los conceptos abordados en la materia Programación 1. La aplicación de listas y diccionarios para representar datos reales, la modularización mediante funciones con responsabilidad única, el manejo de archivos CSV y la implementación de validaciones robustas constituyeron los pilares del sistema.

La experiencia de trabajar en equipo sobre un proyecto funcional reforzó la importancia de la claridad y la legibilidad del código: un código bien comentado y estructurado facilita la colaboración y el mantenimiento futuro. También se valoró el rol de las funciones utilitarias reutilizables, que evitan la duplicación de lógica y reducen la superficie de errores.

Como mejoras futuras se identifican: persistencia en base de datos SQLite, interfaz gráfica con tkinter, exportación de reportes en formato PDF o Excel, y la posibilidad de gestionar imágenes o banderas asociadas a cada país.

4. Bibliografía / Webgrafía

- Van Rossum, G. et al. (2024). Python 3 Documentation. Python Software Foundation. <https://docs.python.org/3/>

- Python Software Foundation. (2024). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (2024). unicodedata — Unicode Database. <https://docs.python.org/3/library/unicodedata.html>
- Lutz, M. (2013). Learning Python (5.^a ed.). O'Reilly Media.
- Matthes, E. (2023). Python Crash Course (3.^a ed.). No Starch Press.
- Real Python. (2024). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- Real Python. (2024). Python's sorted() Function. <https://realpython.com/python-sort/>
- GeeksforGeeks. (2024). Python Dictionary. <https://www.geeksforgeeks.org/python-dictionary/>

Repositorio del proyecto: [https://github.com/\[usuario\]/gestion-paises-tpi](https://github.com/[usuario]/gestion-paises-tpi) **Video**

demostrativo: [Enlace al video — agregar antes de la entrega] **Documentación PDF:**

incluida en la raíz del repositorio como `Informe_TPI_Paises.pdf`